

Code-Layout, Readability and Reusability

Date	29 September 2026
Team ID	SWTID-2026-7995
Project Name	LegalEase AI
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the structure, readability and reusability of the LegalEase AI implementation. The solution uses modular Python files for the Streamlit frontend, FastAPI backend, Gemini generation, document formatting and API testing.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform Python indentation and formatting are used.	Yes	Standard Python indentation.
2	Proper File Structure	Application, API, generation, formatting and test modules are separated.	Yes	Includes app.py, main.py, routes.py, gemini_generator.py and document_formatter.py.
3	Meaningful Variable Names	Names reflect their purpose clearly.	Yes	Descriptive names are used for application logic.
4	Function / Method Names	Functions are descriptively named.	Yes	Operations are separated into functions.
5	Code Comments	Comments clarify important logic where required.	Partial	Comments are used where useful.
6	Modular Design	Code is split into reusable modules.	Yes	Generation and formatting are separate modules.
7	No Redundant Code	Duplicate or unnecessary implementation is avoided.	Yes	Core functionality is organized by module.
8	Error Handling	Expected application/API errors are handled.	Yes	Backend and processing flow include error handling.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Gemini Generator	Python / Gemini	AI document generation workflow	High
2	Document Formatter	Python	TXT, DOCX and PDF export	High
3	FastAPI Routes	Python / FastAPI	API request handling	High
4	Streamlit Application	Python / Streamlit	User input, generation, review and export	Medium